Staff Login Security Features & Security Testing Documentation

Health Database / Multi-Tenant Facility Management System — 9 September 2026

1. Purpose

This document records the security controls implemented for the Staff Login layer and the security/authorization attacks and functional validation tests performed during hardening.

2. Staff Login Security Features

#	Security Feature	Implementation / Security Purpose
1	Firebase Authentication	Staff authentication begins with the facility tenant's Firebase Authentication service; the backend verifies the resulting Firebase ID token.
2	Mother DB Staff Discovery	The login flow resolves the staff email against the Mother DB authorized_clinical_staff profile and obtains the assigned facility context.
3	Server-Side Pending Login Context	The resolved facility and staff email are retained in the server-side pending login context before tenant session establishment.
4	Tenant Firebase Configuration	The backend returns tenant Firebase configuration only after a valid pending login context exists and revalidates the facility against the Mother DB.
5	Tenant Token Verification	The submitted Firebase ID token is verified with the Firebase Admin SDK initialized for the resolved tenant facility.
6	UID / Email Identity Validation	The backend compares token identity with the pending staff identity and re-reads the Mother DB staff profile before creating the session.
7	Server-Side Authorization Data	Clearance, position, facility, and staff identity used for the Flask session are derived from trusted server-side records rather than client-supplied privilege fields.
8	Tenant Firebase Project Binding	The tenant service-account credential's actual project ID is compared with the facility's expected projectId from the Mother DB before the tenant Admin SDK is initialized.
9	Fail-Closed Tenant Initialization	Missing, inactive, invalid, or project-mismatched tenant configuration causes the tenant connection to fail rather than falling back to project-ID-only access.
10	Authenticated Staff Session	After successful tenant verification, the backend creates the Flask staff session with the verified UID, email, facility, clearance, position, and authentication timestamp.

3. Security Attack & Validation Tests

#	Test / Attack	Method	Observed Result	Status
1	Fake Firebase token	Submitted an invalid/fabricated token to /api/staff/session with no valid pending context.	HTTP 401; login session context was required.	PASS
2	Empty session request	Submitted an empty request body to /api/staff/session without a valid pending login context.	HTTP 401; request rejected before tenant authentication.	PASS
3	Direct Firebase-config access	Requested /api/login/firebase-config without a valid pending login context.	HTTP 401; tenant configuration was not disclosed.	PASS
4	Unauthenticated facility spoof	Attempted to influence tenant configuration using facility_id values such as master/lakan_cc without an authenticated pending context.	HTTP 401; request rejected.	PASS
5	Cross-facility config spoof	With pending facility lakan_cc, supplied facility_id=master and another facility to the config endpoint.	Server continued using the pending lakan_cc context and returned the lakan_cc configuration.	PASS
6	Forged identity / privilege fields	Submitted a valid tenant token together with forged UID, email, facility, role, clearance, and position fields.	Server ignored client-supplied identity/privilege fields and returned the trusted tenant values.	PASS
7	Cross-project token isolation	Established pending lakan_cc context, then submitted a valid Mother Firebase (lakan-db) token to /api/staff/session.	HTTP 403; Mother token was rejected by the lakan_cc tenant verification path.	PASS
8	Legitimate tenant authentication	Authenticated using the legitimate lakan_cc staff Firebase account and submitted its tenant ID token.	HTTP 200; facility=lakan_cc, clearance=Admin, position=CMsq.	PASS
9	Tenant project-binding verification	Triggered tenant Admin SDK initialization for lakan_cc and verified the credential against the Mother DB project mapping.	Server logged verified binding: lakan_cc → posh-c013a; session request returned HTTP 200.	PASS
10	Legitimate identity revalidation	Repeated the authenticated session request with a real tenant token while including conflicting client identity values.	Server returned the actual trusted facility/clearance/position values rather than the forged values.	PASS

4. Test Results Summary

10 validation tests were documented. All documented security and authentication tests passed based on observed HTTP responses and server-side behavior.

Category	Tests	Result
Authentication / session boundary	4	PASS
Tenant isolation / facility spoofing	2	PASS
User identity validation	2	PASS
Tenant project binding	1	PASS
Legitimate tenant authentication	1	PASS
Total	10	PASS

5. Security Notes and Limitations

• Testing was performed against the local Staff Login application using localhost endpoints and a legitimate tenant staff account plus test/fabricated request values.

• The documented tests focused on authentication boundaries, tenant selection, cross-project token isolation, identity validation, server-side privilege derivation, and tenant Firebase project binding.

• A PASS records observed behavior at test time; it is not a claim of complete security or a substitute for a formal penetration test.

• The positive tenant-binding test confirmed that lakan_cc was bound to Firebase project posh-c013a before tenant Admin SDK initialization.

• The temporary browser token/debugging instrumentation used during testing was not treated as a production security feature and was subsequently removed from console output.

6. Conclusion

The Staff Login layer demonstrated effective enforcement of tenant-aware authentication, server-side identity validation, cross-project token isolation, server-derived authorization data, and Firebase project binding during the documented local validation tests. The positive tenant-binding test confirmed that the lakan_cc tenant credential was verified against the expected Firebase project posh-c013a before tenant Admin SDK initialization.